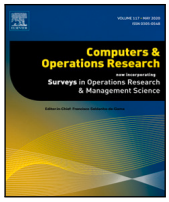
Computers & Operations Research 172 (2024) 106820



Contents lists available at ScienceDirect

Computers and Operations Research

journal homepage: www.elsevier.com/locate/cor





Problem size reduction methods for large CVRPs

Alina G. Dragomir [a,*], David I. Müller [b]

[a] Department of Business Decisions and Analytics, Faculty of Business, Economics and Statistics, University of Vienna, Oskar-Morgenstern-Platz 1, Vienna, 1090, Austria

[b] Institute for Theoretical Physics, TU Wien, Wiedner Hauptstr. 8-10, Vienna, 1040, Austria

ARTICLE INFO

Keywords:
Vehicle routing
Heuristics
Decomposition methods

ABSTRACT

We solve the Capacitated Vehicle Routing Problem (CVRP) by introducing a novel approach to problem size reduction. We propose the generation of short sequences of nodes called "sections", which effectively act as single nodes in a reduced CVRP that is faster and easier to solve. Three section generation methods are compared, and the trade-off between solution quality and computation time savings is evaluated. We show that reduced problem sizes of up to around 60 percent of the original problem size, result in only modest decreases in solution quality, but allow for significant reductions of computation time, regardless of the optimization algorithm used. Our findings highlight the potential benefits of problem aggregation and size reduction for large-scale CVRPs and suggest opportunities for further improving aggregation methods.

1. Introduction

The capacitated vehicle routing problem (CVRP) is a long-established model to solve simplified versions of various real-world logistic problems. It has been applied to a variety of problems, ranging from waste collection to applications in the food industry (Toth and Vigo, 2002). The unceasing demand to solve problems of large scales to better reflect real-world problems, lead to the continuous advancements of solution methods. However, even with modern solution methods and increased computational power, solving truly large-scale problems remains a challenge. This problem becomes increasingly more difficult to solve the larger the problem size, and depending on the selected solution method, requires increasingly more computational time to obtain a solution of a certain quality. Therefore, methods to effectively reduce the problem size, ideally without compromising solution quality, are highly desirable.

Our approach in this paper is to reduce the size of a given CVRP by grouping or clustering customer nodes together in short, incomplete partial routes, which we call *sections*. These sections can be treated as effective nodes for a reduced CVRP, which is easier to solve compared to the original problem. In principle, the clustering of nodes into sections can be done optimally, in the sense that the optimal solution of the reduced problem corresponds to the optimal solution of the original problem. However, it is a priori unknown which nodes should be grouped and in what sequence they should be visited, and we thus have to rely on heuristic methods. The quality of the solution of the reduced problem strongly depends on the heuristic used to generate the sections. Our approach is a type of decomposition method, where, instead of the original CVRP, we first solve the problem of finding a suitable set of sections, and then solve the reduced CVRP.

The idea of problem decomposition, coarsening, or aggregation for a problem size reduction is not new. In some works, the decomposition aims at solving separate sub-problems and merging their solutions, as is the case in Taillard (1993), who apply a purely spatial decomposition, or in Bent and Van Hentenryck (2010), who are able to apply a temporal decomposition as well since their problem has time windows. In general, any two-phase method, also known as a *cluster-first-route-second* approach, is included in this type of decomposition and have already been used for many years. For example, in 1974, Gillett and Miller (1974) introduced the now widely-known sweep algorithm, in Christofides et al. (1979) used a nearest neighbor clustering rule before routing, and in 1981 several different clustering techniques are presented by Fisher and Jaikumar (1981). Nowadays, however, cluster-first-route-second approaches are predominantly used in rich or large vehicle routing problems: Kheiri et al. (2019) use it for a combined scheduling and routing problem, Goetschalckx and Jacobs-Blecha (1989), Koç and Laporte (2018), and Toth and Vigo (1999) apply it to VRPs with backhauls, while Batsyn et al. (2021) apply it to VRPs with split deliveries. This approach is also utilized by Kloimüllner et al. (2015) and Schuijbroek et al. (2017) for inventory rebalancing in bike sharing systems. Dragomir et al. (2022) and Mahmoudi et al. (2019) use it in rich pickup-and-delivery problems. Cluster-first-route-second approaches are still applied and extended in recent works, for

* Corresponding author.
E-mail addresses: alina-gabriela.dragomir@univie.ac.at (A.G. Dragomir), dmueller@hep.itp.tuwien.ac.at (D.I. Müller).

https://doi.org/10.1016/j.cor.2024.106820
Received 23 October 2023; Received in revised form 19 August 2024; Accepted 20 August 2024
Available online 23 August 2024
0305-0548/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

example by Singh et al. (2023) in a cash-collection routing problem from 2023. All the above-mentioned methods attempt to make clusters of sizes that ideally fit exactly one or more vehicles, in contrast to our solution approach where sections are supposed to fill only a partial vehicle, so that multiple sections can be serviced by a single vehicle. Furthermore, in contrast to clusters in literature, our sections have a fixed visit sequence, which can even be reversed in the subsequent optimization algorithm.

Based on the recent work and nomenclature by Santini et al. (2023) we can classify our work as heterogeneous decomposition where we compare several path-based methods (see Section 2.3). In their homogeneous approach, they are able to use the same technique for the main problem and the sub-problem. On the other hand, our method is heterogeneous, since we "require a tailored technique for the solution of the sub-problem" (Santini et al., 2023). This is because the direction of the sequences are not seen as fixed. Similar ideas to our own can be found in the work by Chevalier and Safro (2009) who compare coarsening schemes for multilevel graph partitioning. They aggregate nodes in different magnitudes/degrees so that the original problem is still represented but with fewer degrees of freedom. These approximated solutions are then used to generate a good solution for the original problem. Walshaw (2002) uses a multilevel approach to make clusters out of pairs of nodes and define a new graph, applied to the travelling salesman problem. Blum et al. (2016) extend the matheuristic generate-and-solve framework, where sub-instances of a problem are generated which, when solved and merged, are also solutions to the original instance. The sub-instances are generated heuristically, while an exact method is applied for solving them. The authors improve upon the sub-instance generation and introduce the "construct, merge, solve & adapt" algorithm.

Even though the algorithms in the literature are tested with extensive experimental results, most authors did not evaluate how much a certain degree of aggregation influences either the objective function value or the computation time. Therefore, in this work, our scope is to compare three different decomposition methods for problem size reductions. These are evaluated with respect to solution quality of the reduced problem, solved by a modified Large Neighborhood Search (LNS) (Shaw, 1998). In particular, we investigate the trade-off between the solution quality represented by an objective function value (OFV) and savings in computation time. Lastly, we show the robustness of our findings by using a Hybrid Genetic Search (HGS) algorithm by Vidal (2022) and Vidal et al. (2012) to solve the remaining reduced CVRP. Both LNS and HGS have been proven numerous times to be effective algorithms for solving CVRPs (Petropoulos et al., 2024; Pisinger and Ropke, 2019).

2. Decomposition and problem size reduction for CVRPs

2.1. Problem description

The CVRP is a combinatorial optimization problem with the objective to minimize the total driving distance of a fleet of vehicles, which visit a set of nodes. It is typically defined on a complete graph $G = (N, A)$, with the set $N = \{0, \dots, n\}$ containing all nodes, with node 0 being the depot. All nodes are connected by a directed edge $e \in E = \{(i, j) : i, j \in N\}$. Each edge has an associated travel distance and cost given by d_{ij} corresponding to the distance between the two nodes i and j. In the case of a symmetric problem, the edges are undirected, and the distance matrix D becomes symmetric, $d_{ij} = d_{ji}$. The goal is to find a sequence of visits so that all n customers nodes are visited exactly once. Additionally, the set of identical vehicles must not transport more than their capacity Q and they must start and end their tour at the depot. Given this definition, it is clear that the graph grows quadratically with the number of customer nodes n and the number of possible solutions grows exponentially.

We start our discussion based on the two-index, multi-trip formulation of the CVRP, see Cattaruzza et al. (2016). For each edge (i, j) in the graph G, we define a binary decision variable x_{ij}, where i and j are the start and end nodes, respectively. A vehicle is defined as a sequence of nodes (a route) starting at the depot, visiting a set of customers in a particular order and then returning to the depot. The objective (1) of the CVRP is to minimize the total cost, where c_i is the cost associated with a particular node i. It is customary to set $c_i = 0$, but, in anticipation of the reduced problem, which we discuss below, we keep this term in the total cost function.

$$\min_x C(x) = \min_x \left(\sum_{i \in N} \sum_{j \in N} d_{ij} x_{ij} + \sum_{i \in N \setminus \{0\}} c_i \right) \tag{1}$$

The decision variable x_{ij} is constrained such that each customer node $i > 0$ is visited exactly once, see constraints (2) and (3), and that the quantity of each vehicle, which is the sum of the demands q_i associated with each node along the route, never exceeds the vehicle capacity Q.

$$\sum_{i \in N \setminus \{0\}} x_{ij} = 1, \qquad \forall j \in N, \tag{2}$$

$$\sum_{j \in N} x_{ij} = \sum_{j \in N} x_{ji}, \qquad \forall i \in N. \tag{3}$$

All other constraints are not of particular interest for our discussion, however a complete model can be found in Appendix A. Also, in this work we disregard further constraints such as a maximum number of vehicles and tour length restrictions.

2.2. Reducing problem size via generating sections

Our general idea of reducing the problem size can be classified as a "coarsening and aggregation method" (Santini et al., 2023). In a first step, we attempt to fix a particular edge (a, b) of the graph to create a *section*. Fixing a section amounts to setting $x_{ab} = 1$, and, at the same time, setting all other $x_{ak} = 0$ for $k \in N \setminus \{b\}$ and $x_{kb} = 0$ for $k \in N \setminus \{a\}$ to fulfill the Constraints (2) and (3). Fixing the edge (a, b) eliminates one row and one column from x_{ij} and thus reduces the problem size by one decision. The edge (a, b) can be interpreted as a single effective node c. In this case, we call c a section consisting of two nodes, or equivalently, one edge. The demand associated with c is given by

$$q_c = q_a + q_b \tag{4}$$

and the per node cost is

$$c_c = c_a + c_b + d_{ab}. \tag{5}$$

The elements of the reduced distance matrix $\tilde{D}$ are given by

$$\tilde{d}_{ij} = d_{ij} \qquad \forall i, j \in N \setminus \{a, b, c\}, \tag{6}$$

$$\tilde{d}_{ic} = d_{ia} \qquad \forall i \in N \setminus \{a, c\}, \tag{7}$$

$$\tilde{d}_{cj} = d_{bj} \qquad \forall j \in N \setminus \{b, c\}. \tag{8}$$

Note that the reduced distance matrix must be asymmetric, even if the original problem was symmetric, because our new node is actually a small path, and has its beginning in a different location than its end. That means that if a node is a section, the distance from any node to this section is unequal to the distance from this section back to the node. The cost function of the reduced problem is then defined by

$$\tilde{C}(\tilde{x}) = \sum_{i \in \tilde{N}} \sum_{j \in \tilde{N}} \tilde{d}_{ij} \tilde{x}_{ij} + \sum_{i \in \tilde{N} \setminus \{0\}} c_i, \tag{9}$$

where $\tilde{x}_{ij}$ are the new decision variables and $\tilde{N}$ denotes the set of nodes without a and b, but including the section c. The process of fixing edges, thus merging nodes into sections, can be repeated multiple times until a desired reduced problem size is reached. A small example is shown in Fig. 1.

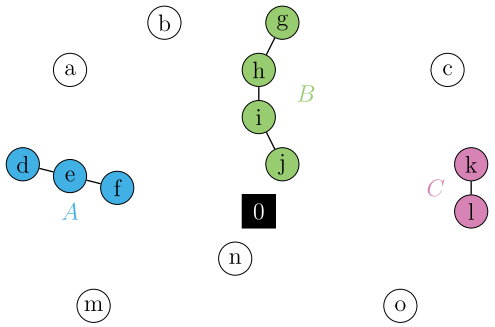


Fig. 1. A small example showing the generation of sections. A problem with 15 nodes (a to o) is reduced to 60 percent of its size. Three sections are generated (A, B, and C) so that only 9 nodes remain in total. Section A contains three nodes (d, e, f) and two edges. Section B contains four nodes (g, h, i, j) and three edges, and section C contains two nodes (k, l) and one edge.

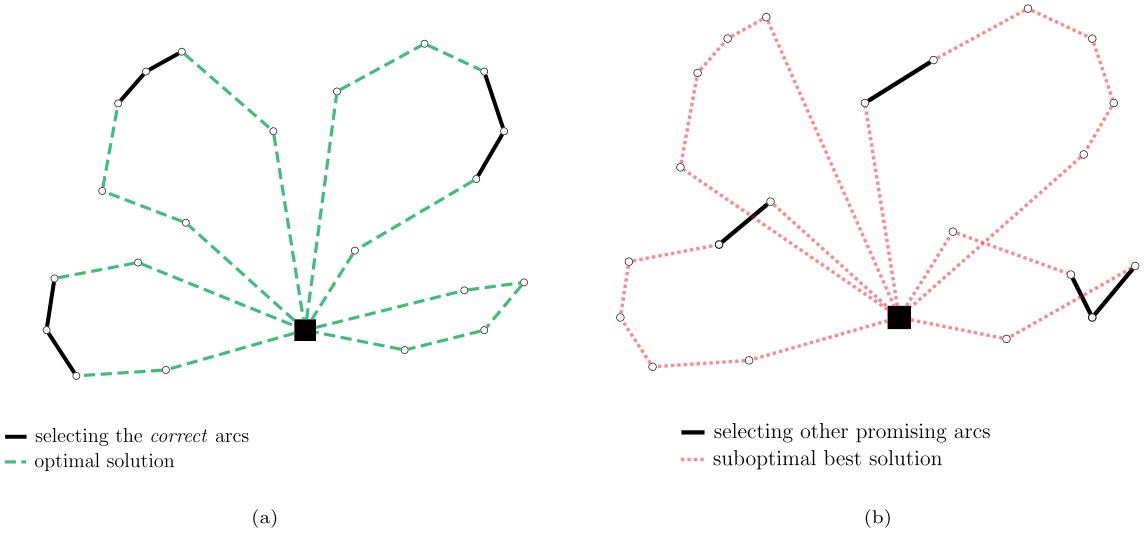


Fig. 2. A small example showing the importance of the selection of edges. In (a) the *correct* edges have been selected, the optimal solution is still achievable. In (b) other promising edges have been selected, however, the optimal solution is no longer achievable.

It is apparent that the reduced problem is structurally the same as the original problem, defined by the objective in Eq. (1). Thus, any standard algorithm to solve CVRPs can be applied *if the sections are assumed to be non-reversible*. If, however, the traversal direction of the sections is a decision variable, a problem-specific algorithm is required. To evaluate this difference, we compare a standard CVRP solver from the literature (HGS (Vidal, 2022; Vidal et al., 2012)) with our own algorithm that allows to reverse the direction of sections.

2.3. Methods for generating sections

The obvious difficulty lies in the selection of the edges to fix. If the *correct* edges are selected, that is, edges that are a subset of the optimal solution, the problem size reduction is not a deterrent for reaching said optimal solution. However, if edges are selected that according to some chosen heuristic appear to be plausible but are not part of the optimal solution, the optimal solution can no longer be achieved, even if the optimal solution of the reduced problem is found. A small illustrative example is shown in Fig. 2. Since it cannot be guaranteed that only the *correct* edges are selected, the question remains of how much the solution quality deteriorates if merely *good* edges are selected.

To generate the sections, three different methods are being compared. The first method, called SE, selects the shortest edges of the problem while upholding some constraints. The method SE_e is an adaptation of SE including a further constraint limiting the number of edges, and is inspired by Santini et al. (2023). The third method NN is based on the Nearest Neighbor algorithm.

All methods use a similar principle, namely selecting single or consecutive edges and merging them to create sections, given some constraints. The order in which edges are considered for sections depends on the method. For the savings-inspired methods SE and SE_e, we first sort the distance matrix from shortest to longest edges. We then fix the edges starting with the shortest until a stopping criterion is reached. This is similar to the savings algorithm, however instead of associating savings $s_{ij} = d_{0i} + d_{j0} - d_{ij}$ to each edge, we simply use the distance d_{ij}. For simplicity, we will continue to call the value *savings* value. In contrast, the NN does not create a sorted list. Instead, we select a random node and merge it with its nearest neighbor. We then select a different random node and continue the process until a stopping criterion is reached. The result of all three methods is a set of sections containing at least one node each, in particular sections consisting of a sequence of multiple nodes.

We introduce a maximum capacity Q_s for the sections, given as a percentage of the vehicle capacity Q. Each new selected edge is only fixed as part of a section if the total demand of all nodes within that section does not exceed Q_s. When selecting edges that would connect nodes that are already part of a section, only the start and end nodes of sections can be selected. This is to avoid a path splitting into two directions. Furthermore, the process of generating sections is stopped as soon as a specific problem size reduction has been achieved. If no

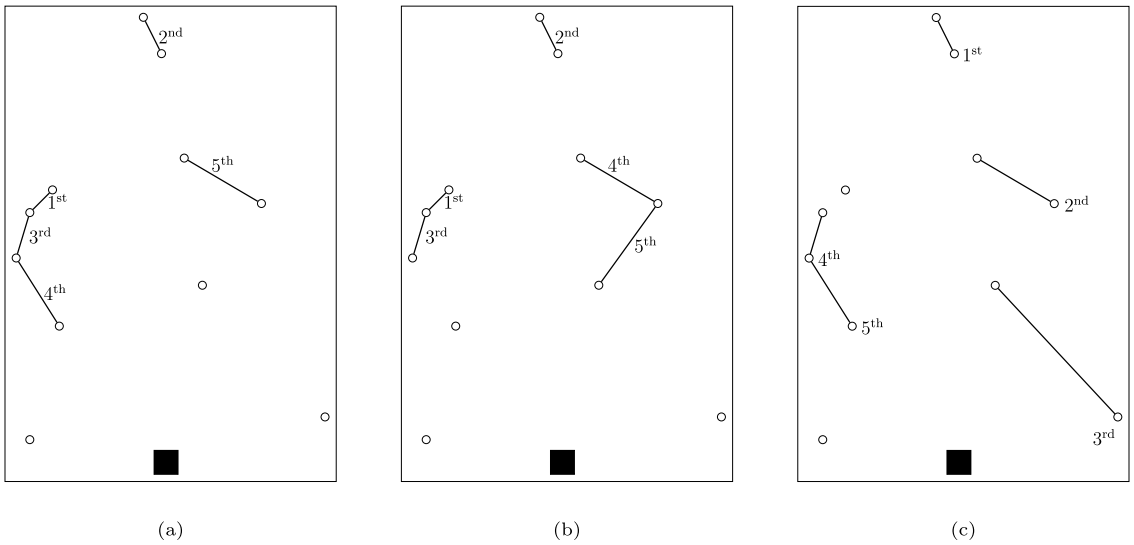


Fig. 3. The difference in selection order for the three section generation methods. (a) shows the order of selection of edges for the SE. (b) shows the order of selection of edges for the SE_e, with $e = 2$, and therefore limiting a section to 2 edges at most. (c) shows the random selection of nodes and the connection to the nearest free neighbor for the NN.

problem size constraint is given, the process continues until all sections have been created that fulfill the capacity constraint. In the case of SE_e, an additional constraint is applied, namely that sections can only include a maximum number of edges. We either allow sections with up to two or three edges, e.g. connecting up to three or four nodes respectively, if no other constraints are violated. Fig. 3 illustrates the differences of the three methods with a small example.

We employ pruning techniques (Arnold et al., 2019; Toth and Vigo, 2003) to speed up the section generation methods and to avoid fixing edges that are less probable in a good solution. When generating the list of savings or when choosing a nearest neighbor, we only consider a certain neighborhood surrounding a node up to a maximum of γ_n neighbors. Of course, even optimal solutions sometimes connect nodes which are far apart (especially in clustered problems), but we attempt to avoid fixing *unlikely* edges in the section generation part of the solver. If those edges would lead to a good solution, they can still be selected as part of the second step, when the reduced problem is solved.

Creating sections can have the same problems as creating (partial) routes with similar algorithms, namely, it can happen that they are crossing themselves, which would make them not optimal (Quintas and Supnick, 1965). We therefore apply a short additional optimization algorithm, that iteratively applies a best-improvement 2-Opt operator (Croes, 1958), and a best-improvement 3-Opt operator (Lin and Kernighan, 1973) for γ_{LS} iterations to avoid sections crossing themselves.

3. Solving the remaining reduced problem

Having chosen a set of sections, our task is to solve the reduced CVRP. Since the reduced problem retains the structure of a CVRP, any standard method can be applied. However, there is one subtlety regarding the direction of the chosen sections that led us to implement a problem-specific operator . When generating sections by any of the three methods, we make a choice on a specific direction for a particular section, namely we fix a beginning and end node. At this stage of the solving process, it is not obvious which of the two possible directions is better or more likely to occur in the optimal solution of the original problem. For asymmetric distance matrices, one could consider which of the two directions is shorter, but in the more common symmetric case, both directions are the same. Thus, it is beneficial to always consider both options through section reversals when solving the reduced CVRP.

In this work, we have implemented a modified LNS (Shaw, 1998) including standard operators, a problem-specific operator for the reversal of the sections, and Simulated Annealing (SA) with oscillating temperatures as a solution acceptance criterion. Our LNS consists of three phases:

(1a) Greedy LNS with υ_{greedy} iterations. Only improvements are accepted.
(1b) Determination of the initial temperature T_s for SA
(2) LNS with SA and an oscillating temperature T_υ for υ_{LNS} iterations
(3) Greedy LNS with υ_{greedy} iterations

An illustration of the phases and the temperature is shown in Fig. 4. Most time is spent on the LNS with SA phase. The individual components are described in detail below. The general algorithm for the LNS loop is depicted in Figure 1, where we start with the initial solution s, which is simultaneously the best solution found so far s^*. An incumbent solution s' gets modified and then either accepted, or discarded, while s^* is always updated if a new best solution has been found.

Solution representation. We adapt a variant of the solution representation of Kytöjoki et al. (2007) with four arrays of length n+1. For the first two arrays (a_{start} and a_{end}), each index represents a route. The array a_{start} contains the starting node of each route after the depot. If it is zero, then the route is empty. a_{end} contains the last node of each route before the depot. For the other two arrays (a_{next} and a_{prev}), each index represents a node. At index i, the array a_{next} contains the next node in the visit sequence after node i. If it is zero, then i is the last node before the depot. a_{prev} contains the previous node in the visit sequence. If it is zero, then i is the starting node of a route. Fig. 5 shows an example of the solution representation for a small problem of 10 nodes. Both the new representation and the more commonly known node sequence are depicted. The main benefit of this solution representation is that it can be stored as arrays of fixed length, which enables low-level code optimizations of certain LNS operators and avoids unnecessary allocation of memory. We note that in principle, it is not necessary to store a_{end} and a_{prev}. However, storing previous nodes enables traversing routes in both forward and backward directions efficiently.

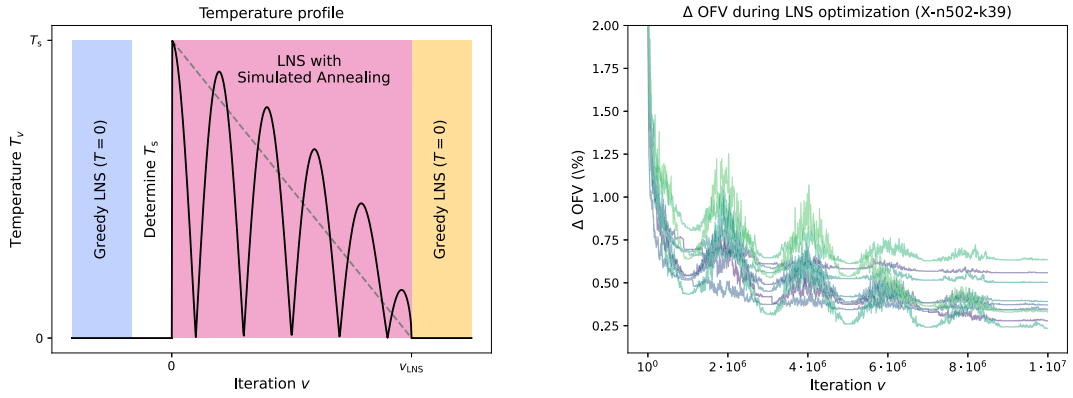


Fig. 4. *(left)* Illustration of the temperature profile and the three phases of our LNS method. *(right)* The total cost of a solution, expressed as the distance to the BKS in percent, during LNS optimization. The oscillating nature of the temperature profile is reflected in the cost as a function of the iteration index. We show ten different random initializations.

Algorithm 1 Large Neighborhood Search (LNS) loop

```
create initial solution s
s* ← s                                              ▷ Initialize best solution found so far s*
while Iteration limit v_LNS not reached do
    procedure Iteration i
        s' ← s                                      ▷ Create incumbent solution s' to be modified
        if i mod v_rs = 0 then                      ▷ Perform the procedure every v_rs iterations
            reverse sections of s'
        else if i mod v_LI = 0 then                 ▷ Perform the procedure every v_LI iterations
            apply local improvement operator to s'
        else
            apply destroy-and-repair operator to s'
        end if
    end procedure
    procedure Use greedy or SA solution acceptance criterion
        if s < s* then                              ▷ A new best solution is found
            s* ← s'                                 ▷ Update the best solution
        end if
        if solution is accepted (SA or greedy) then
            s ← s'                                  ▷ Update the current solution
        else
            s' ← s                                  ▷ Restore previous solution, discard s'
        end if
    end procedure
end while
return s*
```

Index	0	1	2	3	4	5	6	7	8	9	10
a_{start}	0	1	6	0	0	0	0	0	0	0	0
a_{end}	0	10	2	0	0	0	0	0	0	0	0
a_{next}	0	3	0	7	2	9	4	8	5	10	0
a_{prev}	0	0	4	1	6	8	0	3	7	5	9

Route 1: Depot - 1 - 3 - 7 - 8 - 5 - 9 - 10 - Depot
Route 2: Depot - 6 - 4 - 2 - Depot

Fig. 5. A small example showing the solution representation with arrays for a problem with 10 nodes. The two routes, both starting and ending at the depot, are given as a standard node sequence below as well. Note, that index zero is not used, since zero represents the depot. However, since the original representation in Kytöjoki et al. (2007) as well as many programming languages start with index zero, it is included in the solution representation above.

3.1. LNS operators

During an iteration of the LNS one of three operators is applied:

- A destroy-and-repair operator pair, or
- an operator to reverse the direction of sections, or
- a local improvement operator.

Destroy-and-repair operator. Two destroy operators, namely *random removal* and *related removal*, are used, with a destroy rate of τ. Their description can be found in Pisinger and Ropke (2007). Preliminary results indicated that a third destroy operator, *worst removal*, seemed to be detrimental and was therefore not applied for the final results. For repairing the solution, we use a greedy heuristic where a randomized sequence of previously removed nodes are inserted in a new position. All possible positions, including the insertion into an empty route, are attempted. Finally, the insertion is selected by a weighted roulette wheel selection (Bäck, 1996) among the best k positions, with k being a parameter (see Table 1 for all the parameters).

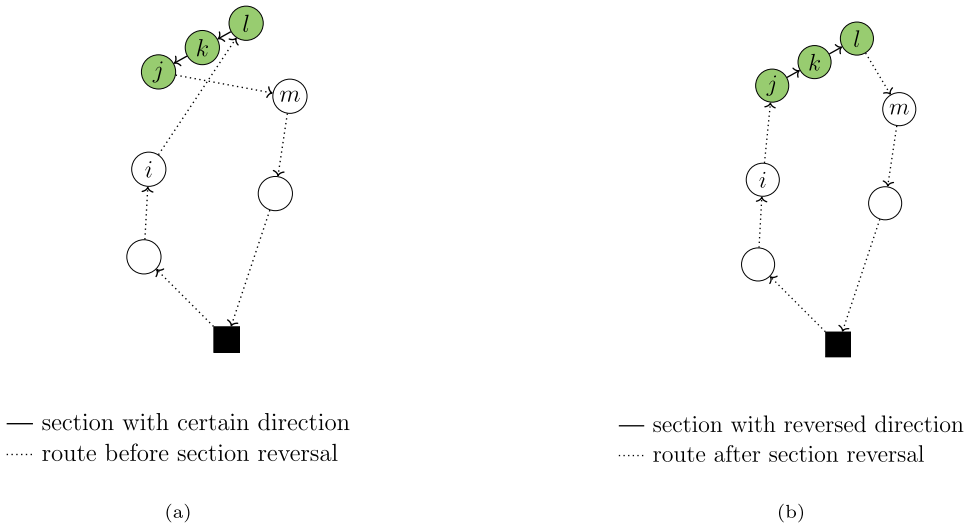


Fig. 6. A small example for the reversal of the direction of a section with three nodes (j, k, l). The route after the reversal is obviously shorter, since the self-crossing is eliminated. The distance from i to the section is d_{ij} before the reversal, and d_{ij} after the reversal. The distance from the section to m changes from d_{jm} to d_{lm}. Section reversal is essential in routes containing more than one section.

Section reversal. The *section reversal* is the main contribution of our algorithm in comparison to standard CVRP solvers. Sections with at least two nodes have different coordinates for entering than for leaving it, and have therefore a determined direction. Reversing the direction can lead to improvements, particularly if sections are visited sequentially or if there is more than one two-node section present in a route. For each section, we swap its direction and determine if this change leads to a reduction in total solution cost. If an improvement is found, the reversed section is kept. Each time a reversal is accepted, the reduced distance matrix must be updated accordingly. In particular, all distances to and from the reversed section, i.e. one row and one column of the distance matrix, must be recomputed. These updates are required because the coordinates of the start and end node of the reversed section have been swapped. Fig. 6 shows an illustration of a section reversal and its benefits.

Local improvement operator. In addition to LNS operators and section reversal, we use two *local improvement operators*: *move sequence* and *swap sequence*, which are applied with equal probability. A local improvement operator is applied instead of a destroy/repair operator within an iteration of the LNS. The *move sequence* operator simply moves a sequence of a certain length to a different position, either within the same route or to a different route. The length $v_{LI_{seq}}$ depends on the average route length in number of nodes of the current solution. The operator *swap sequence* exchanges two sequences of the same length in different vehicle routes. However, the *swap sequence* operator is only applied if the solution contains more than one vehicle tour. For both operators, it is possible that the sequence contains a single node. Both operators are attempted until a feasible option is found that also leads to an improved solution, or until $v_{LI_{max}}$ unsuccessful tries. If no improvement can be found, no move or swap is performed.

3.2. Solution acceptance criterion based on SA

Improved solutions are always accepted as the new incumbent solution s', or even as a new best solution s^*. If a modified solution is worse than the incumbent solution, an adapted SA (Kirkpatrick et al., 1983) framework is used to determine acceptance. A comparison of different acceptance criteria, including SA, and detailed explanations can be found in the study by Santini and Ropke (Santini et al., 2018). In our work, we use the Metropolis–Hastings method to either accept or reject a proposal for a solution which is worse than the incumbent. Given a change in cost of $\Delta C = C_{\text{new}} - C_{\text{incumbent}} > 0$ and a temperature T_v at a given iteration v, we compute $\alpha = \exp(-\Delta C / T_v)$ and compare it to a uniform random number $r \in [0, 1]$. If $r \leq \alpha$ the proposed solution is accepted and rejected otherwise.

We modulate the temperature T_v as a function of the LNS iteration index v. Given a start and end temperature T_s and T_e, we choose our temperature profile to be the geometric mean of a linearly decreasing and an oscillating function:

$$T_v = \sqrt{T_v^{(\text{linear})} T_v^{(\text{osc})}} \tag{10}$$

$$T_v^{(\text{linear})} = T_s + (T_e - T_s)\, v / v_{\text{LNS}}, \tag{11}$$

$$T_v^{(\text{osc})} = T_s + (T_e - T_s)\, \sin^2\left(\pi N_{\text{periods}} v / v_{\text{LNS}}\right), \tag{12}$$

where N_{periods} denotes the number of oscillations and v_{LNS} is the maximum number of LNS iterations. It is easy to see that at the start of the LNS method, we have $T_0 = T_s$ and at the end $T_{v_{\text{LNS}}} = T_e$. The rationale behind choosing an oscillating profile instead of a simple linear decrease, is to reduce the chance of finding local optima. The temperature periodically *reheats* the optimizer, which allows incumbent solutions to admit higher cost values and thus escape possible local minima. The temperature then reduces again to force the incumbent solution into the next local, or hopefully global, optimum. On average, the temperature decreases in such a way that the best found solution improves over time. The idea for an oscillating temperature profile is inspired by Loshchilov and Hutter (2016) and Smith (2017), and preliminary results indicated that an oscillating temperature is superior to a linear decrease.

We determine the initial temperature T_s for each instance separately. After an initial v_{greedy} iterations with zero temperature (only improvements are accepted), we generate v_{prop} proposed solutions and their changes in cost $\{\Delta C_k\}$ with $k \in \{1, 2, \ldots, v_{\text{prop}}\}$. We then set the initial temperature to half of the largest ΔC that was found

$$T_s = \frac{1}{2} \max_k \{|\Delta C_k|\}. \tag{13}$$

The end temperature T_e is set to zero. For a depiction of the oscillating temperature profile as well as a general overview of the phases of our LNS method, we refer the reader back to Fig. 4.

3.3. HGS as an alternative solver

To evaluate the quality of the section-generation methods independently of our LNS/local improvement method, we employ HGS by Vidal (2022) and Vidal et al. (2012) as a second algorithm for comparison.

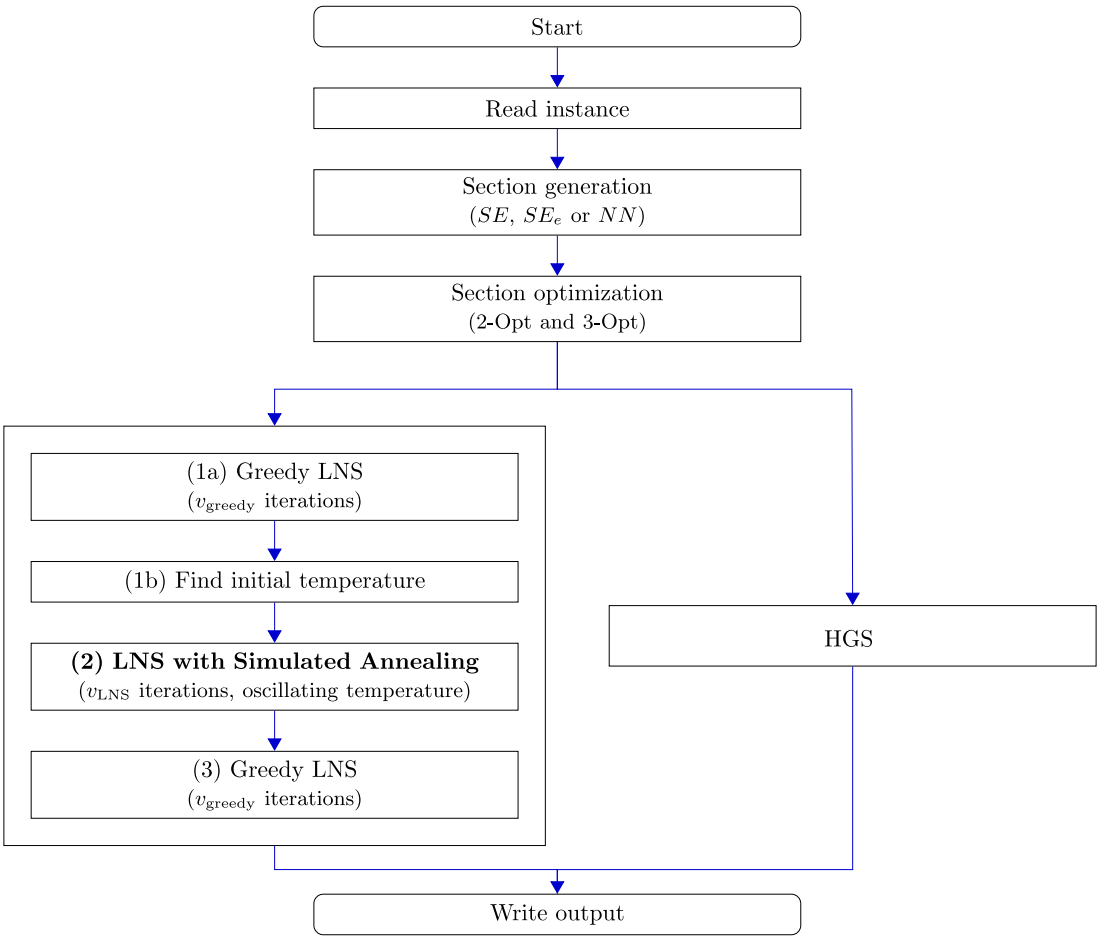


Fig. 7. A flow chart representing our solver framework. After generating and optimizing sections, the reduced problem is either solved by our LNS solver or HGS. Both solvers are allotted the same amount of computation time.

Although HGS is able to deal with asymmetric distance matrices, there is no possibility to reverse the sections. Thus, the essentially random directions of sections obtained by the section generation method are kept. We deem this appropriate for a comparison, since the problem does not have any time windows or other constraints limiting the sequence of visits.

Overview. A flow chart, representing our solution framework, is illustrated in Fig. 7. It shows all major steps while clearly depicting the point where one can choose either between our own LNS implementation or the HGS optimizer from the literature.

4. Computational results

4.1. Instance description

The instances used are the 31 largest X-instances by Uchoa et al. (2017). The instances and their current best-known solution (BKS) are taken from the Capacitated Vehicle Routing Problem Library (CVRPLIB) website.[1] For comparison, the BKS values are reported in our tables.

The instances range in size from 502 to 1001 nodes. They consist of a name, a list of coordinates for a two-dimensional Euclidean plane for each node, a quantity demanded for each node, and a capacity for each vehicle. The instances are very diverse in their demand-capacity-ratio, resulting in some cases in extremely short or long tours in the best known solutions. They do not include a fleet size or tour length restrictions.

4.2. Parameter settings and benchmark setup

The parameters used for the generation of the sections and all parameters used for the LNS implementation can be found in Table 1. These (not necessarily optimal) parameter settings were obtained by a non-exhaustive parameter scan applied to all large X-instances considered in this work. The tested ranges of parameters can be found in Table B.17. All parameters were optimized one by one and in sequence; that is, the best value of the first parameter was determined while fixing the other parameters to reasonable values, before scanning the second parameter and so on. We have used the average ΔOFV (in percent, relative to the BKS) as an optimization metric. The solution algorithms for both section generation and LNS/local improvement have been implemented in Numba 0.50.1 (Lam et al., 2015) and Python 3.7.12. We use the *pyHygese*[2] Python wrapper for the C++ implementation of HGS.[3]

The computations were performed on the Vienna Scientific Cluster 4 (VSC-4) with a CPU of type Intel Xeon Platinum 8174 Processor @ 3.1 GHz. Both our code and HGS were used in single threaded mode.

4.3. Results

We have generated sections with three different methods for reducing the problem size and solved the resulting CVRP with our own LNS

[1] http://vrp.atd-lab.inf.puc-rio.br, accessed on May 5, 2023

[2] version 2.0.0 from https://github.com/chkwon/PyHygese

[3] version 0.0.8 from https://github.com/vidalt/HGS-CVRP

Table 1
Parameters for section generation and LNS.

Parameter	Description	Value
γ_n	max. neighbors to consider when selecting nodes to add to section	3
γ_{LS}	local search iterations for improving sections (move and swap)	10
υ_{LNS}	max. LNS iterations	10^7
υ_{greedy}	max. LNS iterations for greedy search	10^3
υ_{prop}	max. LNS iterations for finding SA starting temperature	10^3
N_{periods}	number of temperature oscillations for SA	5
υ_{LI}	iteration for local improvement	11
υ_{rs}	iteration for section reversal	10
τ	LNS destroy rate range, in percent of problem size	2%–7%
k	best k insertion positions for the repair operator	1, 2, 3
$\upsilon_{LI_{max}}$	maximum tries for attempting local improvement	10
$\upsilon_{LI_{seq}}$	sequence length for local improvement in percent of avg. route length	5%–20%

implementation and the HGS algorithm by Vidal (2022) and Vidal et al. (2012).

For our computational results, we have reduced the problem size of each instance to values between approximately 20 percent and 90 percent of the original problem size.[4] As a baseline, the problem was also solved in its original size. The results in the tables show the deviation in percent Δ (%) to the BKS. We compare the three section generation methods based on the achieved OFV of the overall algorithm (see Section 4.3.1). We then use the most promising method (SE) with the most promising parameter setting ($Q_s = 0.3$) for evaluating the trade-off between computation time and OFV (see Section 4.3.2). We show that a slight decrease in problem size to 0.8 of the original problem size leads to both solutions of slightly better quality, as well as a decrease in computation time of around 25 percent. Further reductions in problem size lead to even smaller computation times but also decreases in solution quality. In addition to our own LNS implementation, we evaluate the same trade-off with HGS, which shows a very similar behavior, which strongly indicates an independence concerning the applied optimization algorithm. Also, we evaluate the OFV by problem size reduction in Section 4.3.3, where it is again evident that a slight reduction leads to better solutions for the LNS. For the HGS we see a slightly different picture, namely that it finds the best solutions without any reduction. It is assumed that this is due to the fact that the HGS solves the general CVRP while our LNS implementation allows the reversal of the sections. Lastly, we report details concerning computation times and run-time limits in Section 4.3.4.

Our overall best results are shown in Table 2 for each instance over multiple runs, the corresponding computation time and the problem size where the best results were found. On average, we obtain a deviation of 1.5 percent, with a median of 1.37 percent, from the BKS. We reach an average deviation of 1.15 percent and a median deviation of 1.2 percent from the BKS with our section generation and HGS.

4.3.1. Comparison of section generation methods

Table 3 show the comparison of the three section generation methods SE, SE_e, and NN. There are no results for a problem size of 0.2, since this amount of problem reduction was not possible for any instance. More detailed results for each problem size separately can be found in Tables B.8 to B.11 in Appendix B. The tables provide both the difference to the best found solution, and the computation time. Each table corresponds to a different problem size, ranging from 0.3 to 0.9.

The results show that NN is consistently worse than the other two methods; however, in some cases, NN is the only method to find any solution at all for some instances with a problem size of 0.3. There seems to be no difference in quality between SE and SE_e, and very little difference in computation time. Although SE and SE_e work differently, with SE limiting the sections generated depending on a factor of vehicle capacity, and SE_e additionally limiting the sections generated depending on the number of edges they are allowed to contain, the edges selected in the sections generated are apparently subsets of one another, which leads to the same solution when applying the LNS. We additionally demonstrate the superior performance of the SE method over NN using a Wilcoxon signed-rank test in Table 4, as proposed by Jastrzab et al. (2024). More specifically, we have used the implementation provided by the Python package SciPy 1.14.0 (Virtanen et al., 2020). The statistical test is based on the relative ΔOFV to the best-known solution provided by the two methods for each instance and reduced problem size. As the null hypothesis, we assume that the two solution methods provide, on average, similar OFVs. The probability distribution for ΔOFV should, therefore, be centered around the same mean value for both methods. Additionally, we assume a symmetric distribution. The resulting low p-values clearly show that the null hypothesis is extremely unlikely to be true and can be rejected at a very low significance level.[5] Thus, we find that SE performs generally better than NN with high statistical significance.

Concerning the parameter for the section generation method SE, different capacity constraints Q_s have been compared in Table 5 for LNS and HGS. For $Q_s = 1.0$, the capacity constraint for this section is equal to the vehicle capacity. The edges are therefore mainly selected by distance. Setting $Q_s = 0.5$ means sections can be generated with a capacity of up to 50 percent of vehicle capacity. If the generated sections saturate this constraint, then only up to two sections can be combined into a route. A Q_s of 0.3 is therefore the most flexible choice, since up to three constraint-saturating sections can be combined during the LNS or HGS. This is further confirmed by our results shown in Tables B.12 and B.13 in Appendix B, where the choice of $Q_s = 0.3$ leads to the best OFVs irrespective of problem size for both LNS and HGS, and where these results are independent of the chosen section generation method.

Concerning the parameter for SE_e that limits the number of edges to include in a section to two or three respectively, we can see the comparison in Table B.15 in Appendix B. For median and average results, a limit of two edges leads to significantly better results (with an average difference of 0.9 percent), while for the best results obtained, a limit of three edges is, on average, slightly better (0.02 percent). The choice of parameter will influence not only the overall OFV, but rather how much the problem can be compressed. Allowing only two edges for each section, that is, connecting up to three nodes, allows the problem to be compressed up to a third of its size and results in a minimum problem size of 0.33. Allowing three edges for each section, and therefore connecting up to four nodes, enables a minimum problem size of 0.25. For the instances solved in our work, a limit of two edges seems to be the better option.

Due to these insights, and with SE having one less constraint and parameter compared to SE_e, we conclude SE to be the best section generation method, and have based subsequent results on this method.

[4] Typically, the reduced problem size relative to the original size is not a round number. For better presentation, we have rounded the reduced problem sizes to the nearest multiple of 10 in percent.

[5] Note that a p-value of $p < 0.05$ indicates that the null hypothesis can be rejected at the 5% significance level.

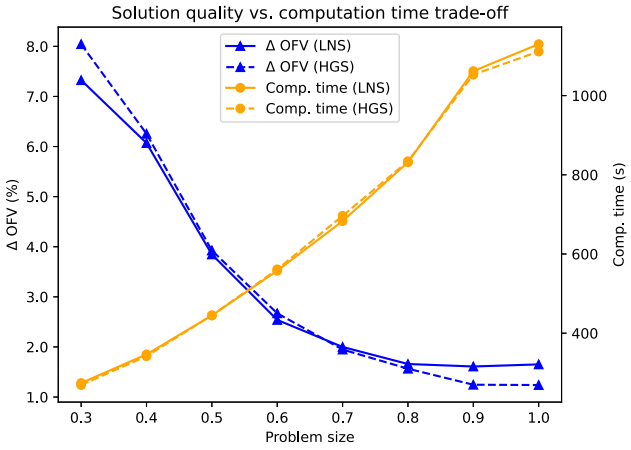


Fig. 8. Trade-off between computation time and solution quality ΔOFV (%) by problem size for both LNS and HGS optimization. For each problem size we selected the best results for each instance, and averaged over all instances.

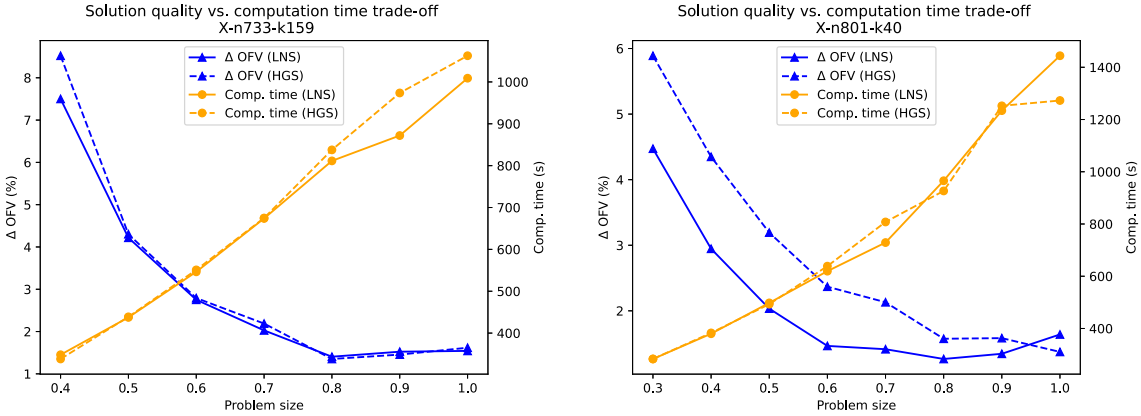


Fig. 9. Trade-off between computation time and solution quality for very diverse instances. Instance X-n733-k159 (left) has a very small ratio between average demand and vehicle capacity, while instance X-n801-k40 (right) has a very large ratio. X-n733-k159 has 733 nodes, a vehicle capacity of 25, and an average demand of 5.4. X-n801-k40 has 801 nodes, a vehicle capacity of 20, and an average demand of 1.

4.3.2. Trade-off between computation time and solution quality

Fig. 8 shows the trade-off between computation time and solution quality by problem size, as an average over the best solution over all instances for both the LNS and HGS method. For the LNS, a significant drop in computation time can be seen from a problem size of 0.8 or smaller. The best solution quality is found at a problem size of 0.9, with 0.8 and 1.0 being only slightly worse. For the HGS, we also see a drop in computation time, however, the solution quality deteriorates for every problem size smaller than 1.0. As stated before, this can be likely attributed to the fact that HGS does not perform section reversals. In general, we can observe a very similar behavior for both algorithms, indicating that our results are independent of the optimization algorithm used. This relationship between computation time and solution quality also holds true for very diverse instances. Fig. 9 shows two example instances: Instance X-n733-k159 has a very small ratio between average demand and vehicle capacity. On average, only five nodes are sequenced for each vehicle, while instance X-n801-k40 has a very large ratio with 20 nodes for each vehicle.

4.3.3. Comparison of OFV by problem size reduction

Tables 6 and 7 show the best results by instance and by problem size for LNS and HGS, respectively. For the LNS, the best results are consistently in the columns with a problem size of 0.8 or 0.9, which is also evident from Table 2 and Table B.14 in Appendix B in which the results are averaged and grouped by node size. It is also interesting to note, that for some instances, a problem size reduction to 0.3 was not possible. The problem size of 0.2 is omitted from the table, since no instance could be reduced by this amount. HGS, on the other hand, provides the best results without any problem size reduction. One possible reason for this is the fact, that HGS cannot reverse sections to find better solutions, in contrast to our LNS implementation. In any case, the results of HGS and LNS are comparable, and show the merit of the problem size reduction independent of the solution algorithm applied to the reduced CVRP.

4.3.4. Computation times

The section generation methods have no hard limit on computation time. The run time instead depends on the chosen method and on the targeted amount of problem size reduction. In terms of computation time for the section generation, SE and SE_e show almost identical results, while NN is slightly faster. The smaller the targeted problem size, the longer it takes to compute the sections, see Fig. 10. We find that the smallest target problem size of 0.3 takes ten times longer than 0.9, but the absolute difference amounts to only around 2.3 s.

The computation times of the subsequent optimization, LNS or HGS, are shown in Table B.16 in Appendix B. We specified the termination criterion for LNS in terms of an upper iteration limit. Since the computation time of a single LNS iteration strongly depends on the problem size, larger instances tend to have overall longer computation times compared to small instances. To enable a fair comparison between HGS and LNS, we used appropriate time limits for HGS such that both algorithms have the same run time. Overall, we find that instances are solved between 404 and 1,013 s.

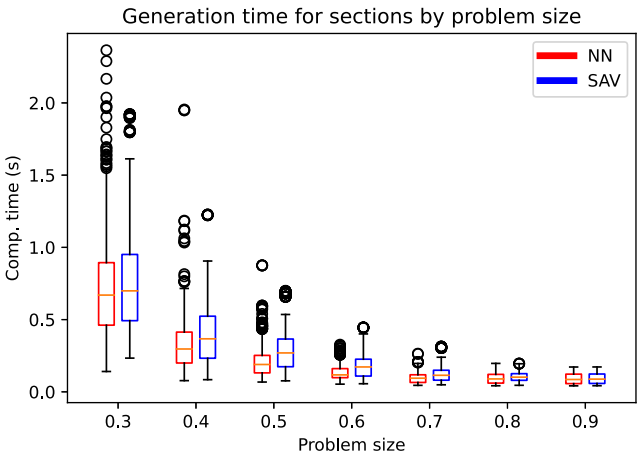


Fig. 10. Comparison of computation time between section generation methods.

Table 2
Best results per instance with LNS and HGS. Instances where the LNS performed better than HGS are marked bold.

Instance	OFV	BKS	Best Δ (%)		Comp. Time (s)		Problem size (%)	
			LNS	HGS	LNS	HGS	LNS	HGS
X-n502-k39	69 388	69 226	0.23	0.26	470	562	0.8	0.9
X-n513-k21	24 720	24 201	2.14	0.00	480	642	0.8	1.0
X-n524-k153	156 694	154 593	1.36	0.01	628	701	0.9	1.0
X-n536-k96	95 882	94 846	1.09	1.12	685	606	1.0	0.9
X-n548-k50	87 175	86 700	0.55	0.61	525	693	0.8	1.0
X-n561-k42	43 459	42 717	1.74	0.57	666	669	0.9	0.9
X-n573-k30	51 205	50 673	1.05	0.61	684	686	0.9	0.9
X-n586-k159	191 920	190 316	0.84	0.86	754	754	1.0	1.0
X-n599-k92	109 723	108 451	1.17	1.40	790	781	1.0	1.0
X-n613-k62	60 995	59 535	2.45	1.54	530	836	0.7	1.0
X-n627-k43	62 957	62 164	1.28	1.36	652	838	0.8	1.0
X-n641-k35	64 558	63 684	1.37	1.37	887	898	1.0	1.0
X-n670-k130	149 701	146 332	2.30	0.48	751	1021	0.8	1.0
X-n685-k75	69 191	68 205	1.45	1.18	1047	910	1.0	0.9
X-n701-k44	82 939	81 923	1.24	1.67	973	1038	0.9	1.0
X-n716-k35	44 098	43 373	1.67	1.25	1125	1018	1.0	0.9
X-n733-k159	138 101	136 187	1.41	1.35	811	837	0.8	0.8
X-n749-k98	78 743	77 269	1.91	2.07	875	1102	0.8	0.9
X-n766-k71	116 185	114 417	1.55	1.15	1179	1097	0.9	0.9
X-n783-k48	73 899	72 386	2.09	1.96	1107	1053	1.0	0.9
X-n801-k40	74 239	73 311	1.27	1.37	965	1272	0.8	1.0
X-n819-k171	160 349	158 121	1.41	1.46	1100	1151	1.0	0.9
X-n837-k142	196 398	193 737	1.37	1.34	1203	1352	0.9	1.0
X-n856-k95	89 956	88 965	1.11	0.72	849	1341	0.7	0.9
X-n876-k59	100 441	99 299	1.15	1.20	853	1462	0.7	0.9
X-n895-k37	55 289	53 860	2.65	2.03	956	1551	0.7	0.9
X-n916-k207	333 335	329 179	1.26	1.05	1479	1543	0.9	1.0
X-n936-k151	136 021	132 715	2.49	0.81	1434	1806	1.0	0.9
X-n957-k87	86 509	85 465	1.22	0.82	1272	1659	0.8	1.0
X-n979-k58	120 380	118 976	1.18	1.52	1722	1247	1.0	0.8
X-n1001-k43	74 188	72 355	2.53	2.38	1346	1861	0.8	1.0
Max.			2.65	2.38	1723	1861	1.00	1.0
Avg.			1.50	1.15	929.48	1064.56	0.87	0.9
Median			1.37	1.20	875.77	1021.44	0.90	0.9
Min.			0.23	0.00	471	563	0.70	0.8

5. Conclusion and outlook

In this work we studied methods to effectively reduce the problem size of CVRPs in a two-phase approach: first, nodes are combined into short paths called sections. This aggregation phase results in a reduced CVRP which has the same structure as the original CVRP albeit smaller, if the sections are considered non-reversible. We compared three different methods to generate sections and solved the reduced problem with our own implementation of LNS, which allows the reversal of sections, and an off-the-shelf implementation of HGS. We compared the solutions of the reduced problems with solutions of the original problem, as well as the best-known solutions from the literature. This enabled us to study the trade-off between solution quality in terms of objective function value and computation time savings for a reduced problem size.

Although we have not studied truly large-scale problems, our results indicate what kind of OFV gap and computational time savings can be expected for problems of much larger size. This work confirms findings from the literature, showing that some decomposition and problem size reduction can even lead to better solutions than solving the original large problem outright. We have seen that the depreciation in OFV is not particularly large for problem size reductions up to 70 or even 60 percent. At the same time, the computation time can be halved. These findings are independent of the original problem size and independent of the optimization algorithm (LNS or HGS). Only for problem size

Table 3
Comparison between section generation methods SE, SE_e and NN by problem size. Best results over all problem sizes with LNS.

Problem size	Δ_{SE} (%)	Δ_{SE_e} (%)	Δ_{NN} (%)
0.3	2.16	2.16	2.37
0.4	1.11	1.11	1.51
0.5	0.68	0.68	1.44
0.6	0.48	0.48	1.05
0.7	0.32	0.32	0.82
0.8	0.23	0.23	0.67
0.9	0.27	0.27	0.47
Max.	2.16	2.16	2.37
Avg.	0.75	0.75	1.09
Median	0.48	0.48	0.94
Min.	0.23	0.23	0.38

Table 4
Statistical analysis for comparing the section generation methods SE and NN using the Wilcoxon signed-rank test. To perform this statistical test, we gather the best OFVs for a given problem size, instance and section generation method. The p-values are obtained by comparing the relative Δ OFVs of the two methods. From the remarkably small p-values we see that SE is clearly outperforming NN in almost all cases. The last two columns show the median (relative) Δ OFV, which also show significant differences at all problem sizes.

Problem size	p-value	med Δ_{SE} (%)	med Δ_{NN} (%)
0.4	3.734×10^{-39}	5.466	6.329
0.5	1.732×10^{-36}	3.975	4.860
0.6	1.825×10^{-40}	3.009	3.733
0.7	2.688×10^{-45}	2.321	3.038
0.8	2.190×10^{-47}	1.900	2.586
0.9	3.965×10^{-28}	1.747	2.054

Table 5
Comparison of our results (OFV gap) of different capacity constraints Q_s for the SE method.

Instance	$\Delta_{Q_s=0.33}$ (%)		$\Delta_{Q_s=0.5}$ (%)		$\Delta_{Q_s=1.0}$ (%)	
	LNS	HGS	LNS	HGS	LNS	HGS
X-n502-k39	0.58	0.85	0.85	1.10	0.77	1.06
X-n513-k21	5.09	4.48	5.20	4.39	5.20	4.41
X-n524-k153	1.81	0.54	3.45	2.40	8.88	8.14
X-n536-k96	1.63	1.86	3.03	2.53	5.25	4.45
X-n548-k50	1.16	1.54	1.51	1.83	1.56	1.83
X-n561-k42	3.57	2.90	4.38	3.78	4.24	3.79
X-n573-k30	2.30	1.78	2.29	2.08	2.39	1.97
X-n586-k159	2.34	2.30	2.34	2.30	4.73	4.15
X-n599-k92	1.89	2.10	2.29	2.49	3.69	3.67
X-n613-k62	3.38	3.53	4.12	4.12	4.42	4.23
X-n627-k43	2.40	2.83	2.47	2.71	2.39	2.65
X-n641-k35	3.08	3.33	3.04	3.30	3.01	3.21
X-n670-k130	3.04	1.43	5.87	4.04	10.20	10.58
X-n685-k75	3.02	2.94	3.32	3.46	4.01	4.69
X-n701-k44	2.50	3.14	2.57	3.12	2.72	3.17
X-n716-k35	3.01	3.15	3.35	3.06	3.44	3.04
X-n733-k159	2.28	2.26	2.95	3.09	3.26	3.44
X-n749-k98	2.59	2.92	3.21	3.38	3.90	3.90
X-n766-k71	2.92	2.75	2.89	2.80	3.89	3.63
X-n783-k48	3.39	3.73	3.53	3.93	3.64	3.89
X-n801-k40	2.23	3.15	2.17	3.06	2.17	3.02
X-n819-k171	1.95	2.09	2.30	2.36	4.67	4.66
X-n837-k142	1.64	1.76	2.00	2.25	3.68	3.47
X-n856-k95	1.36	1.40	1.66	1.72	2.11	2.12
X-n876-k59	1.94	2.44	1.99	2.45	1.99	2.51
X-n895-k37	3.94	4.43	3.98	4.34	3.98	4.38
X-n916-k207	1.46	1.41	1.65	1.60	3.34	3.17
X-n936-k151	3.46	2.42	5.01	3.65	9.03	8.87
X-n957-k87	1.73	2.24	2.22	2.52	2.23	2.58
X-n979-k58	1.84	2.46	2.13	2.50	2.30	2.28
X-n1001-k43	3.63	4.48	3.67	4.47	3.67	4.46
Max.	5.09	4.48	5.87	4.47	10.20	10.58
Avg.	2.49	2.54	2.95	2.93	3.90	3.92
Median	2.34	2.44	2.89	2.80	3.67	3.63
Min.	0.58	0.54	0.85	1.10	0.77	1.06

Table 6
Best OFV gap over all seeds, per instance and per problem size for the LNS algorithm. Best values for each row are highlighted in boldface.

Instance	Problem size							
	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
X-n502-k39	2.16	1.11	0.68	0.48	0.32	**0.23**	0.27	0.38
X-n513-k21	10.89	7.26	5.44	3.67	3.68	**2.14**	2.31	2.87
X-n524-k153	–	15.85	13.28	5.08	2.34	1.72	**1.36**	1.65
X-n536-k96	13.31	9.12	5.97	3.51	1.98	1.35	1.13	**1.09**
X-n548-k50	3.75	2.41	1.56	0.90	0.66	**0.55**	0.69	0.57
X-n561-k42	9.07	6.07	4.26	2.83	2.63	2.22	**1.74**	1.77
X-n573-k30	4.37	3.12	2.35	1.89	1.12	1.25	**1.05**	1.26
X-n586-k159	–	13.05	5.81	3.85	2.89	1.73	1.06	**0.84**
X-n599-k92	9.15	5.46	3.66	2.31	1.95	1.41	1.36	**1.17**
X-n613-k62	7.34	5.81	4.57	3.48	**2.45**	2.51	2.46	2.48
X-n627-k43	4.97	3.23	2.39	1.58	1.55	**1.28**	1.35	1.30
X-n641-k35	6.48	4.30	2.90	2.26	1.68	1.69	1.41	**1.37**
X-n670-k130	–	17.22	8.45	3.90	3.24	**2.30**	2.70	2.91
X-n685-k75	8.89	5.83	4.20	2.79	2.29	1.82	1.73	**1.45**
X-n701-k44	4.68	3.56	2.55	1.81	1.59	1.44	**1.24**	1.40
X-n716-k35	5.83	4.00	2.86	2.28	2.06	1.90	2.04	**1.67**
X-n733-k159	–	7.50	4.22	2.75	2.03	**1.41**	1.52	1.54
X-n749-k98	8.20	5.58	3.85	2.98	2.32	**1.91**	1.99	2.25
X-n766-k71	10.21	4.98	3.45	2.64	2.07	1.75	**1.55**	1.66
X-n783-k48	5.68	4.63	3.57	2.62	2.33	2.28	2.11	**2.09**
X-n801-k40	4.47	2.95	2.03	1.46	1.41	**1.27**	1.34	1.64
X-n819-k171	11.72	7.72	4.50	3.24	2.28	1.63	1.43	**1.41**
X-n837-k142	10.86	5.13	3.24	2.09	1.57	1.43	**1.37**	1.43
X-n856-k95	5.23	2.67	1.79	1.43	**1.11**	1.14	1.12	1.18
X-n876-k59	3.91	2.40	1.71	1.56	**1.15**	1.18	1.30	1.46
X-n895-k37	7.59	5.02	3.38	2.69	**2.65**	2.75	2.98	2.69
X-n916-k207	–	8.25	4.61	2.61	1.93	1.39	**1.26**	1.34
X-n936-k151	17.30	14.78	4.58	4.00	3.21	2.79	2.74	**2.49**
X-n957-k87	4.50	2.73	2.01	1.52	1.29	**1.22**	1.30	1.59
X-n979-k58	3.28	2.10	1.71	1.44	1.49	1.22	1.40	**1.18**
X-n1001-k43	6.60	4.28	3.70	3.00	2.69	**2.53**	2.57	3.04
Max.	17.30	17.22	13.28	5.08	3.68	**2.79**	2.98	3.04
Avg.	7.32	6.07	3.85	2.54	2.00	1.66	**1.61**	1.65
Med.	6.54	5.02	3.57	2.62	2.03	1.63	**1.40**	1.46
Min.	2.16	1.11	0.68	0.48	0.32	**0.23**	0.27	0.38

reductions to 50 percent or smaller, the deviation of OFV from the BKS increases significantly. This increase goes hand in hand with a decrease in computation time to around a third or a fourth of the original problem. Depending on the required goal, whether to favor a better OFV or a shorter computation time, our work gives some indication about the potential of aggregating the problem and reducing the problem size.

Future work should focus on improved reduction methods. We have shown that our results are approximately independent of the solution algorithm for the reduced problem. Thus, the performance of our approach is largely limited by the quality of the selected sections. Any improvements to the edge selection heuristic would have a significant impact on overall solution quality.

The methods presented in this work (SE, SE_e and NN) are very simple and could be extended by taking into account typical features of (almost) optimal solutions, for example the typical orientation of edges near and far away from the depot, average route lengths, customer location and demand distributions, and so on. On the other hand it seems feasible that this type of problem is well-suited for a machine learning classification or ranking model, which could extract the most useful features for selecting optimal edges automatically by training on large datasets of best known solutions. Beyond mere improvements of the section generation method, one could also consider an iterative approach: instead of selecting edges once at the beginning and then only solving the reduced problem, it would be interesting to refine the selection of edges during the main optimization procedure. If certain sections appear to hinder optimality, they could be split or reordered into a new set of sections. It should also be noted that our current reduction methods are very fast compared to how much time is spent

Table 7
Best OFV gap over all seeds, per instance and per problem size for the HGS method. Best values for each row are highlighted in boldface.

Instance	Problem size							
	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
X-n502-k39	2.82	1.52	1.03	0.80	0.57	0.42	**0.26**	0.28
X-n513-k21	11.90	6.86	5.10	3.03	2.03	1.14	0.38	**0.00**
X-n524-k153	–	15.64	13.01	4.38	1.03	0.42	0.16	**0.01**
X-n536-k96	11.90	6.97	4.50	2.71	1.94	1.66	**1.12**	1.13
X-n548-k50	4.18	2.64	1.97	1.35	0.88	0.77	0.65	**0.61**
X-n561-k42	9.75	6.23	4.02	2.81	1.70	0.85	**0.57**	0.76
X-n573-k30	4.24	3.15	2.16	1.50	1.18	0.88	**0.61**	0.62
X-n586-k159	–	11.11	4.91	3.54	2.59	1.75	1.02	**0.86**
X-n599-k92	8.87	5.06	3.46	2.62	2.06	1.83	1.57	**1.40**
X-n613-k62	8.28	6.17	4.67	3.14	2.10	1.67	1.76	**1.54**
X-n627-k43	4.93	3.60	2.77	2.15	1.87	1.59	1.45	**1.36**
X-n641-k35	6.45	4.57	3.42	2.62	2.06	1.40	1.55	**1.37**
X-n670-k130	19.14	17.28	7.16	2.57	1.59	0.96	0.58	**0.48**
X-n685-k75	11.39	6.96	4.48	2.93	2.22	1.91	**1.18**	1.67
X-n701-k44	5.74	4.06	2.94	2.50	2.23	2.07	1.69	**1.67**
X-n716-k35	6.28	3.98	3.12	2.53	2.08	1.75	**1.25**	1.32
X-n733-k159	–	8.52	4.30	2.79	2.19	**1.35**	1.46	1.62
X-n749-k98	7.62	5.27	3.72	3.25	2.39	2.48	**2.07**	2.23
X-n766-k71	8.99	4.83	3.42	2.64	1.99	1.61	**1.15**	1.50
X-n783-k48	7.06	5.06	4.05	3.13	2.89	2.76	**1.96**	2.13
X-n801-k40	5.89	4.35	3.19	2.37	2.13	**1.57**	1.58	1.37
X-n819-k171	10.82	7.69	4.52	3.48	2.30	1.93	**1.46**	1.57
X-n837-k142	8.76	4.46	3.15	2.57	2.05	1.71	1.48	**1.34**
X-n856-k95	5.14	3.01	2.17	1.72	1.04	**0.72**	**0.72**	0.75
X-n876-k59	4.22	3.46	2.51	2.00	1.63	1.52	**1.20**	1.45
X-n895-k37	8.47	6.11	4.39	3.63	2.72	2.59	**2.03**	2.09
X-n916-k207	–	7.29	4.04	2.84	2.06	1.45	1.24	**1.05**
X-n936-k151	18.39	16.19	4.16	3.13	2.28	1.73	**0.81**	1.39
X-n957-k87	4.80	3.70	2.64	2.19	1.66	1.37	1.07	**0.82**
X-n979-k58	3.59	2.71	2.29	1.86	1.59	**1.52**	1.64	1.67
X-n1001-k43	7.60	5.47	4.43	4.01	3.34	3.12	2.94	**2.38**
Max.	19.14	17.28	13.01	4.38	3.34	3.12	2.94	**2.38**
Avg.	8.04	6.25	3.93	2.67	1.95	1.57	1.25	**1.24**
Med.	7.60	5.06	3.72	2.64	2.06	1.59	**1.24**	1.37
Min.	2.82	1.52	1.03	0.80	0.57	0.42	0.16	**0.00**

on solving the reduced problem using LNS or HGS (1–2 s compared to 400 to 1000 s). More sophisticated extensions of our methods could be allotted more computational budget in the overall algorithm to allow for optimizing large-scale problems without significant deterioration in solution quality.

CRediT authorship contribution statement

Alina G. Dragomir: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **David I. Müller:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization.

Data availability

Data will be made available on request.

Acknowledgments

This research was funded in whole or in part by the Austrian Science Fund (FWF) [Grant-DOI: 10.55776/P34455 and 10.55776/P34764]. The computational results presented have been achieved in part using the Vienna Scientific Cluster (VSC). We thank Karl F. Doerner for insightful discussions and constructive criticism.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cor.2024.106820.

References

Arnold, Florian, Gendreau, Michel, Sörensen, Kenneth, 2019. Efficiently solving very large-scale routing problems. Comput. Oper. Res. 107, 32–42. http://dx.doi.org/10.1016/j.cor.2019.03.006.

Bäck, Thomas, 1996. Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms. Oxford university press, ISBN: 0-19-509971-0.

Batsyn, Mikhail V., Batsyna, Ekaterina K., Bychkov, Ilya S., Pardalos, Panos M., 2021. Vehicle assignment in site-dependent vehicle routing problems with split deliveries. Oper. Res. 21, 399–423. http://dx.doi.org/10.1007/s12351-019-00471-7.

Bent, Russell, Van Hentenryck, Pascal, 2010. Spatial, temporal, and hybrid decompositions for large-scale vehicle routing with time windows. In: International Conference on Principles and Practice of Constraint Programming. Springer, pp. 99–113. http://dx.doi.org/10.1007/978-3-642-15396-9_11.

Blum, Christian, Pinacho, Pedro, López-Ibáñez, Manuel, Lozano, José A., 2016. Construct, merge, solve & adapt a new general algorithm for combinatorial optimization. Comput. Oper. Res. 68, 75–88. http://dx.doi.org/10.1016/j.cor.2015.10.014.

Cattaruzza, Diego, Absi, Nabil, Feillet, Dominique, 2016. Vehicle routing problems with multiple trips. 4OR 14, 223–259. http://dx.doi.org/10.1007/s10288-016-0306-2.

Chevalier, Cédric, Safro, Ilya, 2009. Comparison of coarsening schemes for multilevel graph partitioning. In: International Conference on Learning and Intelligent Optimization. Springer, pp. 191–205. http://dx.doi.org/10.1007/978-3-642-11169-3_14.

Christofides, Nicos, Mingozzi, A., Toth, Paolo, 1979. Combinatorial optimization. John Wiley & Sons Ltd, ISBN: 0471997498.

Croes, Georges A., 1958. A method for solving traveling-salesman problems. Oper. Res. 6 (6), 791–812. http://dx.doi.org/10.1287/opre.6.6.791.

Dragomir, Alina G., Van Woensel, Tom, Doerner, Karl F., 2022. The pickup and delivery problem with alternative locations and overlapping time windows. Comput. Oper. Res. 143, 105758. http://dx.doi.org/10.1016/j.cor.2022.105758.

Fisher, Marshall L., Jaikumar, Ramchandran, 1981. A generalized assignment heuristic for vehicle routing. Networks 11 (2), 109–124. http://dx.doi.org/10.1002/net.3230110205.

Gillett, Billy E., Miller, Leland R., 1974. A heuristic algorithm for the vehicle-dispatch problem. Oper. Res. 22 (2), 340–349. http://dx.doi.org/10.1287/opre.22.2.340.

Goetschalckx, Marc, Jacobs-Blecha, Charlotte, 1989. The vehicle routing problem with backhauls. European J. Oper. Res. 42 (1), 39–51. http://dx.doi.org/10.1016/0377-2217(89)90057-X.

Jastrzab, Tomasz, Myller, Michal, Tulczyjew, Lukasz, Blocho, Miroslaw, Kawulok, Michal, Czornik, Adam, Nalepa, Jakub, 2024. Standardized validation of vehicle routing algorithms. Appl. Intell. 1–30. http://dx.doi.org/10.1007/s10489-023-05212-0.

Kheiri, Ahmed, Dragomir, Alina G., Mueller, David, Gromicho, Joaquim, Jagtenberg, Caroline, van Hoorn, Jelke J., 2019. Tackling a VRP challenge to redistribute scarce equipment within time windows using metaheuristic algorithms. EURO J. Transp. Logist. 8 (5), 561–595. http://dx.doi.org/10.1007/s13676-019-00143-8.

Kirkpatrick, Scott, Gelatt, Jr., C. Daniel, Vecchi, Mario P., 1983. Optimization by simulated annealing. Science 220 (4598), 671–680. http://dx.doi.org/10.1126/science.220.4598.671.

Kloimüllner, Christian, Papazek, Petrina, Hu, Bin, Raidl, Günther R., 2015. A cluster-first route-second approach for balancing bicycle sharing systems. In: Computer Aided Systems Theory–EUROCAST 2015: 15th International Conference, Las Palmas de Gran Canaria, Spain, February 8–13, 2015, Revised Selected Papers 15. Springer, pp. 439–446. http://dx.doi.org/10.1007/978-3-319-27340-2_55.

Koç, Çağrı, Laporte, Gilbert, 2018. Vehicle routing with backhauls: Review and research perspectives. Comput. Oper. Res. 91, 79–91. http://dx.doi.org/10.1016/j.cor.2017.11.003.

Kytöjoki, Jari, Nuortio, Teemu, Bräysy, Olli, Gendreau, Michel, 2007. An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. Comput. Oper. Res. 34 (9), 2743–2757. http://dx.doi.org/10.1016/j.cor.2005.10.010.

Lam, Siu Kwan, Pitrou, Antoine, Seibert, Stanley, 2015. Numba: A LLVM-Based Python JIT Compiler. In: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC. LLVM '15, Association for Computing Machinery, New York, NY, USA, http://dx.doi.org/10.1145/2833157.2833162.

Lin, Shen, Kernighan, Brian W., 1973. An effective heuristic algorithm for the traveling-salesman problem. Oper. Res. 21 (2), 498–516. http://dx.doi.org/10.1287/opre.21.2.498.

Loshchilov, Ilya, Hutter, Frank, 2016. SGDR: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983.

Mahmoudi, Monirehalsadat, Chen, Junhua, Shi, Tie, Zhang, Yongxiang, Zhou, Xuesong, 2019. A cumulative service state representation for the pickup and delivery problem with transfers. Transp. Res. B 129, 351–380. http://dx.doi.org/10.1016/j.trb.2019.09.015.

Petropoulos, Fotios, Laporte, Gilbert, Aktas, Emel, Alumur, Sibel A., Archetti, Claudia, Ayhan, Hayriye, Battarra, Maria, Bennell, Julia A., Bourjolly, Jean-Marie, Boylan, John E., et al., 2024. Operational research: methods and applications. J. Oper. Res. Soc. 75 (3), 423–617. http://dx.doi.org/10.1080/01605682.2023.2253852.

Pisinger, David, Ropke, Stefan, 2007. A general heuristic for vehicle routing problems. Comput. Oper. Res. 34 (8), 2403–2435. http://dx.doi.org/10.1016/j.cor.2005.09.012.

Pisinger, David, Ropke, Stefan, 2019. Large neighborhood search. In: Gendreau, Michel, Potvin, Jean-Yves (Eds.), Handbook of Metaheuristics. Springer International Publishing, Cham, pp. 99–127. http://dx.doi.org/10.1007/978-3-319-91086-4_4.

Quintas, Louis V., Supnick, Fred, 1965. On some properties of shortest Hamiltonian circuits. Amer. Math. Monthly 72 (9), 977–980. http://dx.doi.org/10.2307/2313333, URL http://www.jstor.org/stable/2313333.

Santini, Alberto, Ropke, Stefan, Hvattum, Lars Magnus, 2018. A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. J. Heuristics 24, 783–815. http://dx.doi.org/10.1007/s10732-018-9377-x.

Santini, Alberto, Schneider, Michael, Vidal, Thibaut, Vigo, Daniele, 2023. Decomposition strategies for vehicle routing heuristics. INFORMS J. Comput. http://dx.doi.org/10.1287/ijoc.2023.1288.

Schuijbroek, Jasper, Hampshire, Robert C., van Hoeve, Willem-Jan, 2017. Inventory rebalancing and vehicle routing in bike sharing systems. European J. Oper. Res. 257 (3), 992–1004. http://dx.doi.org/10.1016/j.ejor.2016.08.029.

Shaw, Paul, 1998. Using constraint programming and local search methods to solve vehicle routing problems. In: International Conference on Principles and Practice of Constraint Programming. Springer, pp. 417–431. http://dx.doi.org/10.1007/3-540-49481-2_30.

Singh, Bismark, Oberfichtner, Lena, Ivliev, Sergey, 2023. Heuristics for a cash-collection routing problem with a cluster-first route-second approach. Ann. Oper. Res. 322 (1), 413–440. http://dx.doi.org/10.1007/s10479-022-04883-1.

Smith, Leslie N., 2017. Cyclical learning rates for training neural networks. In: 2017 IEEE Winter Conference on Applications of Computer Vision. WACV, IEEE, pp. 464–472, arXiv:1506.01186.

Taillard, Éric, 1993. Parallel iterative search methods for vehicle routing problems. Networks 23 (8), 661–673. http://dx.doi.org/10.1002/net.3230230804.

Toth, Paolo, Vigo, Daniele, 1999. A heuristic algorithm for the symmetric and asymmetric vehicle routing problems with backhauls. European J. Oper. Res. 113 (3), 528–543. http://dx.doi.org/10.1016/S0377-2217(98)00086-1.

Toth, Paolo, Vigo, Daniele, 2002. The Vehicle Routing Problem. SIAM, ISBN: 0898714982.

Toth, Paolo, Vigo, Daniele, 2003. The granular tabu search and its application to the vehicle-routing problem. INFORMS J. Comput. 15 (4), 333–346. http://dx.doi.org/10.1287/ijoc.15.4.333.24890.

Uchoa, Eduardo, Pecin, Diego, Pessoa, Artur, Poggi, Marcus, Vidal, Thibaut, Subramanian, Anand, 2017. New benchmark instances for the capacitated vehicle routing problem. European J. Oper. Res. 257 (3), 845–858. http://dx.doi.org/10.1016/j.ejor.2016.08.012.

Vidal, Thibaut, 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. Comput. Oper. Res. 140, 105643. http://dx.doi.org/10.1016/j.cor.2021.105643.

Vidal, Thibaut, Crainic, Teodor Gabriel, Gendreau, Michel, Lahrichi, Nadia, Rei, Walter, 2012. A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. Oper. Res. 60 (3), 611–624. http://dx.doi.org/10.1287/opre.1120.1048.

Virtanen, Pauli, Gommers, Ralf, Oliphant, Travis E., Haberland, Matt, Reddy, Tyler, Cournapeau, David, Burovski, Evgeni, Peterson, Pearu, Weckesser, Warren, Bright, Jonathan, van der Walt, Stéfan J., Brett, Matthew, Wilson, Joshua, Millman, K. Jarrod, Mayorov, Nikolay, Nelson, Andrew R.J., Jones, Eric, Kern, Robert, Larson, Eric, Carey, C.J., Polat, İlhan, Feng, Yu, Moore, Eric W., VanderPlas, Jake, Laxalde, Denis, Perktold, Josef, Cimrman, Robert, Henriksen, Ian, Quintero, E.A., Harris, Charles R., Archibald, Anne M., Ribeiro, Antônio H., Pedregosa, Fabian, van Mulbregt, Paul, SciPy 1.0 Contributors, 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. Nat. Methods 17, 261–272. http://dx.doi.org/10.1038/s41592-019-0686-2.

Walshaw, Chris, 2002. A multilevel approach to the travelling salesman problem. Oper. Res. 50 (5), 862–877. http://dx.doi.org/10.1287/opre.50.5.862.373.